

P4 programmability for fault-tolerant Software Defined Flying ad-hoc Network

Domenico Uomo^a ,^{*} Andrea Sgambelluri^a, Loyal Ismail^a, Francesco Paolucci^b

^a Scuola Superiore Sant'Anna, Via G. Moruzzi 1, Pisa, 56124, Italy

^b CNIT, Via G. Moruzzi 1, Pisa, 56124, Italy

ARTICLE INFO

Keywords:

FANet
P4-language
UAV
SDN
Reliability

ABSTRACT

This paper introduces the issue of highly available *Software Defined Flying ad-hoc Network* for critical real-time applications. FANETs are a particular implementation of VANET (Vehicle Ad-hoc NETWORKs) where the vehicles, in the specific case, are Unmanned Aerial Vehicles (UAV), or drones. VANETs in general suffer from the mobility of the nodes, which can lead to continuous disconnections and topology changes. This issue becomes particularly troublesome for Flying ad-hoc networks (FANET), which are characterized by flying nodes, with low node density and a high rate of topology changes. On the other hand, nowadays, drones are widely used both in civil and industrial environments, and thus they may need to satisfy certain requirements in terms of the availability of communication. In the traditional distributed networks, the propagation of the information when there is a change in the topology, and the consequential update of the information in each switch, can introduce long periods of unavailability, which cannot be tolerated in certain contexts. For this reason, we decided to introduce an approach based on SND to manage FANET in an efficient and reliable way. The SDN architecture relies on a centralized controller to configure the network devices by exploiting a control plane channel. However, having a centralized controller implies a single point of failure. Moreover, control plane links are critical in SDN, since the controller uses those links to update the configuration of the switches. In order to efficiently introduce the SDN paradigm in a dependable FANET it is worth considering a novel approach, where the controller is responsible for the main routing decisions, but the network is able to operate autonomously and tolerate topology changes, within a certain degree, even when the SDN controller is not connected. The proposed architecture is a P4-SDN, that is capable of automatically rerouting traffic even when the controller cannot communicate with the involved devices, providing then an additional level of redundancy. The programmable switches are instructed to take countermeasures when an unexpected link fault occurs and the controller is unreachable. In our solution, the controller configures both primary and backup actions for each expected flow, leveraging the programmability of P4-enabled devices. Thus, each switch can choose to forward the packets over the primary path or follow the backup action. Since these countermeasures are taken locally and independently from the controller, this approach allows the network to operate even when the control-plane link is unusable. It also improves the handover of the fault when the controller is reachable, minimizing critical application traffic disruptions.

1. Introduction

Flying ad-hoc Networks (FANETs) are becoming a widely used solution for many kinds of applications. Thanks to the flexibility of Unmanned Aerial Vehicles (UAV) networks, and the different types of drones, FANETs can be an interesting solution both in civilian and military contexts [1]. While deploying this kind of network, it must be taken into account that flying objects can continuously change the shape of the topology through wireless connections and disconnections

due to the mobility of the UAVs. In such a scenario, the nodes of the network need to run routing algorithms frequently, bringing an overhead in the communication and a waste of bandwidth. One solution can be the implementation of an SD-FANET where the controller is responsible for finding the shortest path among two hosts.

While implementing an SDN, that decouples the data-plane and control plane, we must consider that a link fault due to the movement of a UAV is possible, but also a control-plane link fault is possible. In such a scenario, not only the node might not be able to communicate

^{*} Corresponding author.

E-mail address: domenico.uomo@santannapisa.it (D. Uomo).

<https://doi.org/10.1016/j.comnet.2025.111572>

Received 13 February 2025; Received in revised form 27 June 2025; Accepted 17 July 2025

Available online 26 July 2025

1389-1286/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

with the former neighbors, but it may not even be reconfigured since it becomes unreachable from the controller. The consequences of such behavior strictly depend on the kind of application that is using the network. If the network is used by general-purpose applications, then we can experience a loss in the quality of service, but, if critical real-time applications need to communicate and synchronize, then the whole system can be invalidated.

This paper aims to provide a methodology that allows the deployment of an SD-FANET in a way that significantly improves the availability of the network, reducing the recovery time and providing backup mechanisms in case the switches become unreachable and thus no longer configurable.

For this purpose, we propose the adoption of a programmable data plane with P4-enabled switches as network devices taking advantage of the programmability features they offer. This choice allows us to program them with actions and countermeasures for events of link and/or node failure, even though the controller is unreachable and, thus, the control plane is temporarily not reconfigurable.

The paper is organized as follows:

1. An introduction of the P4-FANET considered as our case study;
2. The description of the SDN-Controller we developed for this scenario, including the extended path computation algorithms to find backup ports in each switch and the libraries used to configure the P4-switches properly;
3. Scalability and capacity test of the controller to ensure that it is able to handle a proper amount of requests on networks of different sizes;
4. The experimental results on a real testbed, where we demonstrate an improvement of the packet loss and the availability of the network.

2. Background and related work

Several studies have been carried out focusing on the dependability and QoS of Mobile Networks. As pointed out in [2], one of the key elements to reach acceptable Quality of Service (QoS), especially for mobile networks, is the capability of distributing and updating policies on the fly, to adapt the architecture to a changing environment. Particularly, in our case study, we identify an SDN controller as the provider of such capabilities. This approach raises the issue of the dependability of the SDN. For instance, in [3], the authors discuss the challenges of SDN. According to them, the main concern is about the consistency and the availability of the controller. Moreover, a survey is presented about the possible errors that can occur in such a network and the failures that can lead to. Many approaches have been proposed to improve the dependability of SDN. In [4], both the dependability and security threats are discussed. In that paper, the proposed approaches concern the replication of the controller, diversity and self-healing mechanisms. These approaches improve the robustness of the controller, overcoming the problem brought by a centralized architecture, but they can raise several additional issues. The most relevant one is about the consistency of the information stored by different replicas, given the limitation from the CAP theorem [5]. In [6], the authors issued the problem of dependability and security of the network using blockchains. Thus, the control plane is decentralized thanks to the presence of more than one controller. The consistency of the operations is implicitly guaranteed by the blockchain. The limit of this approach lies in the efficiency of the blockchain, which can bring a high transaction latency and low throughput [7]. On the other hand, if network devices are mobile objects like UAVs, the switches of the network can lose connectivity with the controller, even if they are both healthy. This raises a new challenge: deploy the controllers in a way that maximizes the coverage area. The solution to this problem becomes more complex whenever it is impossible to predict the movement patterns of the UAVs. For this reason, it is crucial to grant a certain degree of independence to the switch of the network. One way to enforce that is to deploy ad-hoc routing protocols.

2.1. Routing protocols in FANET

Many routing protocols have been proposed to overcome the issue of FANETs. They can be roughly divided into four subsets [8]:

- static;
- reactive;
- proactive;
- hybrid.

In the next subsections, each routing protocol is discussed in detail.

2.1.1. Static routing protocols

Using static routing protocols, the routing tables are loaded and they cannot be changed until the end of the mission [9]. They are obviously the simplest possible approaches, but they cannot guarantee some kind of fault tolerance to the network. This kind of protocol can work very well when no network changes are expected in the topology [10].

2.1.2. Proactive routing protocols

The proactive routing protocols periodically update the routing tables. This is an efficient way to always have an available path between the source and the destination. However, at the same time, they have the counter-effect of flooding the bandwidth for the updates. However, the efficiency of those protocols has been proven to vary based on the movement pattern of the UAVs [11,12].

2.1.3. Reactive routing protocols

The reactive routing protocols update the routing tables only when a new path computation is requested. In this case, there is no need for a periodic check, and the network does not risk overflowing. However, the user can experience a delay in the communication. Indeed, this kind of protocol implies a greater latency if compared with static and proactive protocols [8,9].

2.1.4. Hybrid routing protocols

The hybrid routing protocols mix the reactive and proactive approaches. They are mainly used for large networks [9], which can be subdivided into more local area networks. Usually, they mix proactive protocols for the routine among nodes in the same zone and reactive protocols for routing among nodes in different zones [13].

2.2. Backup rules for failure recovery in SDN

All the aforementioned routing protocols provide a way to recover from a link failure. At any rate, maintenance is either reactive or proactive, and sometimes, it is a combination of both; however, all of those approaches lead to a packet loss whenever a failure occurs. Our approach to face the fast-changing FANET topology is based on the usage of backup paths pre-computed and pre-configured in the switches of the network. Many approaches have been proposed to deploy similar architectures; however, we believe they do not fit well with the case of a FANET. For example, the authors in [14] proposed a method that allows the user to provide the nodes of the network with backup rules. The proposed algorithm, however, does not consider the case of a switch that, after detecting a failure, does not have any other alternatives to reach the destination, and thus needs to rebound the packets to the destination. For a network with few nodes and links, this could be a very common case, introducing several useless hops in the path between source and destination, becoming a major issue in terms of performance after a failure. The authors in [15,16], on the other hand, made one step forward, considering also the case of packets bounced back. The handling of this case is managed through the usage of labels, and/or the usage of complex algorithms (Depth-First search). Despite being a suitable approach, it introduces an additional layer of complexity and overhead. In our solutions, we propose a mechanism

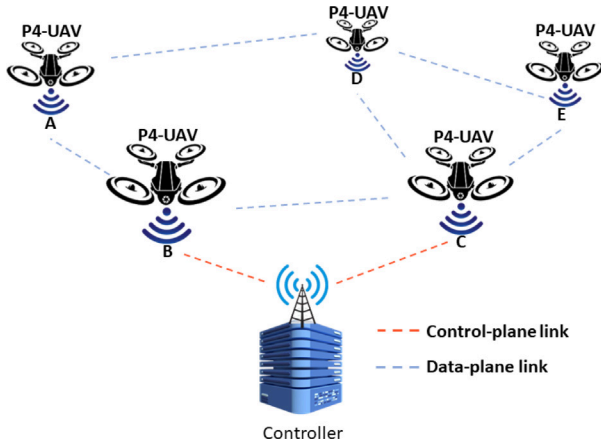


Fig. 1. Example of a Software Defined FANET, where the controller is located at the ground station edge node.

that is based on a status register, leveraging the P4 programmability to access, read, and write memory registers, and make decisions based on the read value. We demonstrate how we can achieve good performance without introducing complexity. Finally, in [17], the authors proposed a method based on hybrid networks and the usage of the IGP protocol to handle failovers. This implementation requires a hybrid network where the Controller is used for global decisions, while the switches can make local decisions. In our approach, in contrast, we still use an SDN, where paths are precomputed, eliminating the time needed to handle a failover.

All the considered approaches managed to reach zero packet loss after failure detection, but they all appear to have significant drawbacks when dealing with fast-changing networks. Besides the aforementioned issue, indeed, it must also be considered that, in a FANET, it is also possible the creation of new links, due to the movement of the UAVs. The proposed architectures not seem to deal with this characteristic.

2.3. Contribution

In our previous work [18], we proposed an approach based on SDN and In-band Telemetry generated by P4-enabled devices and used to feed an AI model able to anticipate disconnection. In this work, we extended the previous approach, integrating it with a solution based on a steering mechanism. The steering enables the possibility to define for each node a backup port when a path request is submitted to the controller. In this way, when a link fault happens, the unavailability time will be given only by the detection time, since the nodes already have an available local alternative path. Thus, we aim to integrate programmable SDN to a FANET, taking into account also the unique characteristics of this kind of network. Like the approaches proposed in [15–17], we manage to have zero packet loss after the detection of the failure. Moreover, our solution uses the backup path only for a defined period of time after failure. The controller recomputes the shortest path and reconfigures the switch as soon as possible, ensuring the optimal path for each flow. Besides, it also considers the case of link creation, a circumstance that is not considered in other solutions, but it is crucial in the FANET scenario, where the movement of UAVs can also lead to this event. In this case, the switches of the network are able to automatically detect a new neighbor and interact with the controller to ask whether new routes are possible.

3. P4 FANET architecture

A P4 FANET network architecture mainly consists of two essential pillars: an edge node hosting the controller, responsible for path

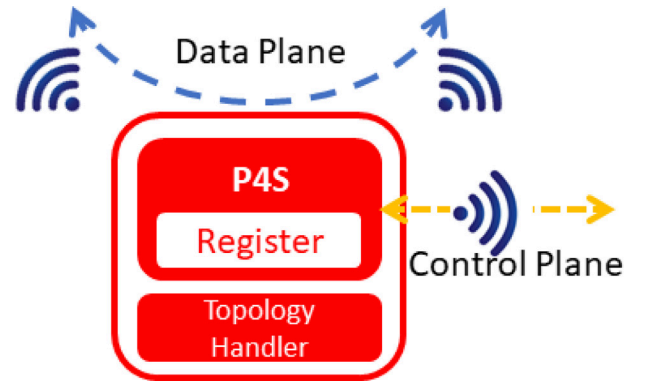


Fig. 2. P4-UAV internal architecture.

computation, and a set of P4-UAVs, as illustrated in Fig. 1. The P4-UAVs form a data-plane (DP) topology, identified by blue links, used to exchange traffic data related to applications. In addition, each P4-UAV is connected to the controller using control-plane (CP) links (i.e., red links).

In the considered scenario, each node can lose the connection with the controller, becoming unreachable. An example is when the controller loses connectivity with the nodes A, D and E. Thus, it is fundamental that the controller programs in advance the switches, allowing the data-plane to react independently in case of link failures. For this purpose, we choose to use P4-switches. The programmability of this kind of device enables the possibility of configuring a set of preventive actions.

In this study, we assume that the SDN controller is located on the edge node at the ground station and is responsible for configuring the P4-UAVs through the control plane (CP) link. CP links are logical connections between the SDN controller and the P4-UAV node and are assumed to be active and connected at the bootstrap. This last assumption is not strictly necessary. P4 switches indeed could be pre-loaded with the correct P4 Pipeline by another entity before joining the network. This would require a second process that manages this task. For simplicity, we assume that the loading of the P4 pipeline is performed by the controller itself.

4. P4 UAV

The P4-UAV in Fig. 2, represents one aerial node in the network and is interconnected through radio links established using standard wireless technology in ISM Band. These links work as transparent L2 connections. An overlay solution is employed by utilizing VXLAN tunneling for Ethernet over IP encapsulation and this approach can be implemented across different radio technologies, as long as they support Ethernet Layer 2 access. The status of the radio links is updated by the Link Layer Discovery Protocol (LLDP) [19], primarily achieving the discovery of the network topology. In this study, the implementation of LLDP has been decentralized to the P4-UAV switches to efficiently manage the dynamic nature of the wireless network, which can pose challenges for an unstable centralized control.

Moreover, the P4-UAV is equipped with a module called *Topology Handler*, which aims to detect new connections on the interface of the data-plane.

4.1. P4-switch

The P4-Switch (P4S) stands as the essential component within the P4-UAV FANET node, enabling the integration of programmable forwarding alongside other drone capabilities such as mobility and mesh connectivity.

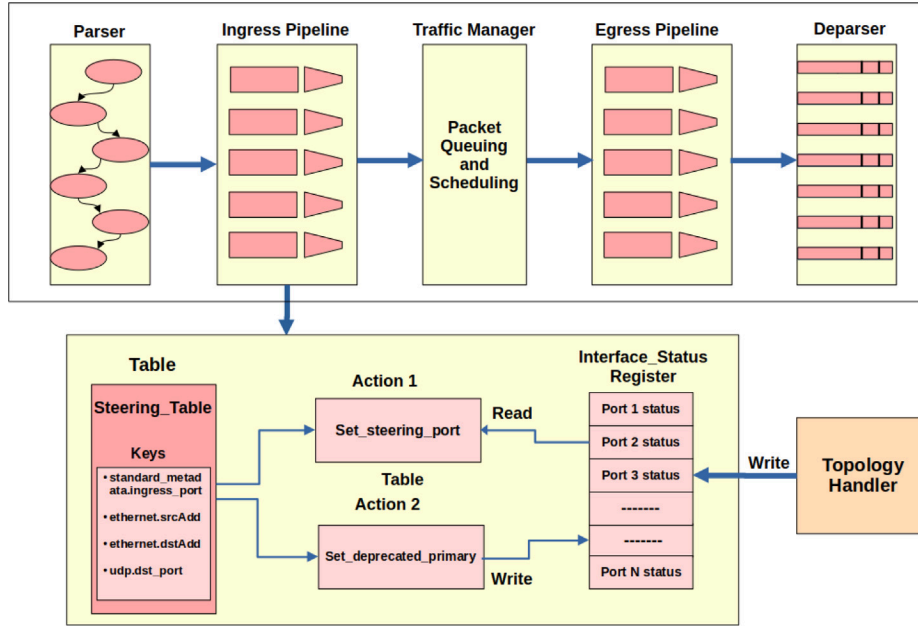


Fig. 3. P4 pipeline architecture.

The internal infrastructure of the P4 switch chosen for this implementation is the V1 architecture, especially the Behavioral Model version 2 (BMv2). This model, shown in Fig. 3, involves the presence of a single parser, an ingress pipeline, a traffic manager, an egress pipeline and a deparser. The parser is responsible for receiving the incoming data from any interface connected to the switch and analyzing it to determine its structure. This involves converting the data into a form of a finite-state machine for further processing in the next stage. Once the data has been parsed by the parser, it is then directed to the ingress pipeline, which comprises sequential match action tables. Here, the data undergoes classification and processing, specifically routing to the appropriate egress pipeline based on matching with different fields. The traffic manager, the only non-programmable P4 module, performs scheduling, packet queuing and load balancing. Its role is to ensure maximum efficiency in traffic management. The egress pipeline, structured similarly to the ingress pipeline, comprises sequential match action tables that allow additional packet processing, such as implementing security measures and filtering policies on outgoing data. Finally, the deparser is responsible for the correct protocol stacking of the outgoing packet. The V1 abstract model describes the interconnection and placement of the various sections but does not define their internal implementation, which is delegated to the P4 program.

4.2. Topology handler

Given the dynamism of the aforementioned scenario, it is crucial for the nodes of the network to be aware of their neighbors and be capable of signaling whenever a link goes down or a new connection is established. This is one of the main differences between the deployed SDN architecture and the traditional one. In particular, in the proposed approach, the discovery functionality is managed by the data-plane devices and not by the central controller. To achieve this goal, each node of the network is equipped with a software module called Topology Handler, which is used for the topology discovery. This module leverages the LLDP protocol to announce the topology details (i.e., node ID and port ID) to discover new neighbors. The module is able to exchange REST messages towards the controller in case of new links and to report possible link failures. During its execution, the module periodically checks for interfaces up and running. Whenever a new interface is available, the topology handler starts two threads: a

Sender and a *Receiver*. The Receiver thread works in listening mode on the new interface, waiting for LLDP packets. At the same time, the Sender thread starts to send LLDP packets to discover new neighbors. Whenever an LLDP packet is received by a Receiver thread, the node assumes that another switch is connected to that interface. By considering the transmitted information, the Topology Handler calls a REST API towards the SDN controller, notifying the details of the newly established connection. After the discovery of a new link, in order not to consume unnecessary bandwidth, the Sender thread sends LLDP packets for other 5 s and then it stops. In this way, no bandwidth and energy are consumed for the transmission of LLDP probe packets. The connection details are recorded in a local register. Moreover, the Topology Handler module monitors the status of the interfaces. In case the status of an interface becomes no longer available (i.e., the interface goes down), the Topology Handler retrieves the connection details from the local register and notifies the link-down event to the SDN controller relying on its exposed REST API. The link-up and link-down events are transmitted to the SDN controller by both the edges of the link. In this way, even in case the controller is not reachable by one of the two switches, the topology change event received by the other one successfully updates the topology (i.e., each link is considered bidirectional by default).

4.3. P4 steering

In normal P4 solutions, the traffic is forwarded from a source to a destination following the computed path. In this case, the flow is configured targeting the high reliability, by identifying and configuring, in each traversed switch, a primary output port and a backup output port, as demonstrated in Fig. 4.

The decision on the output port to be used is performed locally, depending on the operational status of the ports, which can be:

- UP: The link connected to this interface is active and has no failures.
- DOWN: The link connected to this interface has failed.
- UP_DEPRECATED: A link connected to a subsequent node in the path has failed.

For example, considering the flow between hosts h1 and h2, the primary path (1–2–3–4–7) is considered, while, for each node on the

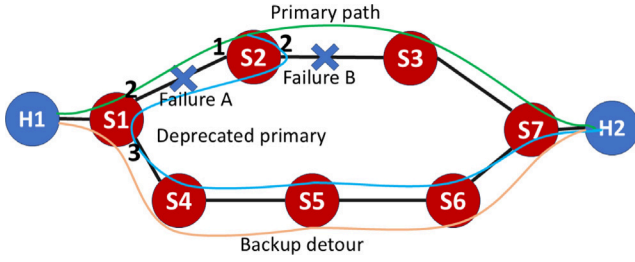


Fig. 4. P4 Steering.

primary path, a backup detour is computed. In the figure, only the backup detour related to node S1 (4–5–6–7) has been drawn. In the example, node S1 is configured with port 2 as primary and port 3 as backup. When a packet arrives to node S1, the primary port is adopted for the forwarding only if its status is UP. When failure A occurs, on the link between S1 and S2 depicted in Fig. 4, the state of port 2 of the S1 node will be set to DOWN and the packets will be diverted through port 3 along the backup path. Considering the event of failure B shown in figure Fig. 4, the failure occurs in the link between S2 and S3. The packet will follow the primary path until reaching S2, where port 2 is down. S2 will then follow the previous mechanism and reroute the packet through its local backup detour in a bounce-back manner towards S1. In this case, an intermediate status level of the interface has been defined as ‘UP_DEPRECATED’. This state indicates that the port is active but its use is deprecated as there is a failure along the forwarding path. The steering policy implemented in this case is as: upon the occurrence of the first bounce back event, the match of the packet entering from port 2 of S1 triggers the forwarding towards the local backup detour and updates the status of port 2 from ‘UP’ to ‘UP_DEPRECATED’. This implies that subsequent packets of the flow arriving on S1 from port 1 will find their primary path inhibited by the new state of port 2 and will be directly forwarded on the backup detour, even if the connected interface is physically available. The previous steering policies have been implemented in P4 using the following elements shown in Fig. 3:

1. A P4 register, called *interface_status*, contains the updated operational status of the data plane interfaces, including UP, DOWN and, UP_DEPRECATED. This register consists of N fields where N corresponds to the maximum number of wireless interfaces in the p4 switch. Since BMv2 and P4-language do not directly support visibility of the status of the data plane interfaces connected to the P4S, the ‘DOWN’ status and ‘UP’ status events are updated by the Topology Handler through external writes to the register. While the ‘UP_DEPRECATED’ status is updated internally within the switch when specific events occur (bounce back event).
2. A flow table, called *steering_table*, defined in the ingress pipeline. The matching in this table is achieved by considering the ingress port, MAC addresses, and UDP destination port. Depending on this matching, one of two actions is invoked:
 - *Set_steering_port*: responsible for setting the output port between two candidate ports from the flow rules, assigned based on the state of the ports themselves read from the *interface_status* register;
 - *Set_deprecated_primary*: responsible for setting the output port for the bounced back packet and updating the relevant port status in the *interface_status* register for the correct forwarding of the subsequent packets.

If we consider, for instance, the switch S1 from Fig. 4, the schema of flow table “steering table” is shown in 1

Table 1

Schema of the steering table in the P4 switch.

Src	Dst	Ing port	Action	arg
h1	h2	1	Set steering ports	(2, 3) ^a
h1	h2	2	Set deprecated primary	(2, 3) ^a

^a Respectively, primary port and backup port.

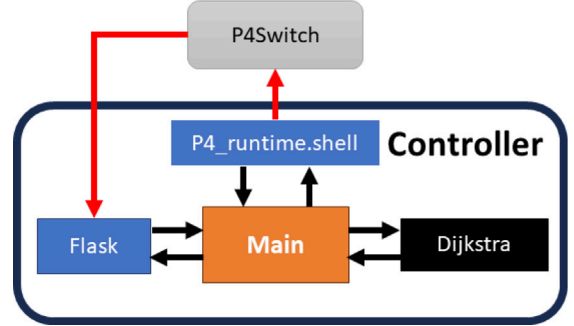


Fig. 5. Design of the SDN controller.

5. SDN controller

The P4 FANET configuration is performed by an ad-hoc SDN Controller which uses several modules to perform its task. All the different modules are used by the core engine which is responsible for calling the different functions based on the state of the execution. At the bootstrap, the core engine opens an interface used by the P4 switches called *Topology Update Interface*. Through this interface, the nodes of the network can communicate the presence of a new link or the disruption of an old one. When a Topology update request is received by the controller, the core engine updates a graph that represents the network topology through a *graph management* module. The same interface is used to receive a path computation request. When this happens, the core engine uses again the graph management module to compute the shortest path between the two nodes. Eventually, if a path is available, the core engine uses its southbound interface through the P4 switches to configure the flow rules for each of them.

5.1. Controller implementation

The controller described in the previous section is entirely written using Python3.8. The core engine is the main part of the application that was developed. It takes advantage of the *p4runtime.shell* library [20] to implement the southbound interface and write the P4-switches’ routing tables. At the same time, the Topology Update interface is deployed using API implemented through Flask [21]. Finally, the path computation module uses the Dijkstra algorithm to find the shortest path between two end-points that need to communicate. The schema of the P4-Controller is shown in Fig. 5

The steering table is then used by the P4 switch as shown in the pseudocode in 1

In order to properly perform its tasks, the controller needs to meet the following prerequisites:

- A JSON file with all the switches of the network, with their IP and MAC address, is provided;
- A JSON file with all the hosts of the network with the MAC address and the end-point is provided;
- The p4-pipeline (JSON file) that must be installed into the P4-enabled switches;
- All the switches, at the bootstrap, are connected to the control plane;

Algorithm 1: Steering algorithm

```

1 Input: primary, backup
2 metadata  $\leftarrow$  parse(packet)     $\triangleright$  We parse the packet to obtain
   source and destination. Also, we retrieve the ingress port.
3 match(header, table)             $\triangleright$  Check the action to perform
4 if Action = set_steering_port then
5   output  $\leftarrow$  primary
6   status  $\leftarrow$  Interface_status_registry(primary)
7   if status = 0 OR status = 1 then
8     output  $\leftarrow$  backup
9   end
10  if Action = set_deprecated_primary then
11    output  $\leftarrow$  backup
12    Interface_status_registry(primary)  $\leftarrow$  1     $\triangleright$  Code for
       deprecated
13  end
14 end
15 return output

```

- All and only the connections among the switches are provided to the controller through the REST-API it exposes.

Particularly, this last requirement is due to the fact that the controller does not make any assumption about the connections of the network. This way, the controller can adapt to any network topology communicated by the switches.

At the bootstrap, the controller reads the details of each switch of the network and starts creating a graph. In this stage, the graph is composed only of nodes without any edges (i.e., links). Then, the controller opens a shell through each known switch and installs the P4-pipeline provided at the beginning, then it waits for the switches to call the Rest-API.

At this stage, the API messages can be of three different kinds:

- Link Up
- Link Down
- Compute Path

The Rest-API of *Link Up* notifies the controller of the presence of a new link in the network, while the *Link Down* API notifies the controller of the disruption on a link. The *Compute Path* API is needed to ask the controller to compute the shortest path among two hosts of the network with the related backup routes. For each Link Up or Link Down API received, the controller updates the graph that it stores in memory. This graph is then used both to compute the shortest path whenever a Compute Path request arrives and to recompute each previously requested path after a Link Down Request arrives.

5.2. Graph management

The topology graph is managed through a module called *dijkstra.py*. This module defines the class **Graph** and the class **Vertex**. The class graph contains a hash table of the vertex composing the graph itself. Moreover, each vertex contains another hash table that stores its own neighbors, and the port where each neighbor is connected.

This module implements the Dijkstra Algorithm to find the shortest path. The implementation of the Dijkstra algorithm uses, rather than an array to store the node list, a data structure called *heapq* that behaves like a binary tree. In this way, it optimizes the search of the nodes. The complexity of the search thus, is $O((V + E) \times \log_2 V)$

The input parameters are the source node (src), destination node (dst) and the graph where the computation must be performed (graph). Note that every time a shortest path is requested, the distances of the nodes must be re-initialized

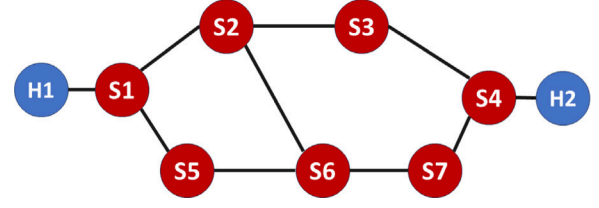


Fig. 6. Topology example.

5.3. Rest-API implementation

Another fundamental part of the P4-Controller is the implementation of the Rest-API. This interface is necessary in order to allow the components of the network to communicate with the controller about network variations and path computation requests. The interface API are all deployed using the Flask library.

The call of the APIs occurs through the HTTP's method *get*. The URL is then parsed to find out the operations and the parameters necessary to execute that operation. Particularly, the allowed operations are the following:

- Link Up: needs the parameters switch source, switch destination, source port and destination port
- Link Down: needs the parameters switch ID, defined in the JSON file describing the nodes of the network, and port ID
- Compute Path: needs the parameters switch source and switch destination

Whereas the controller receives a Compute Path request, it will proceed to compute the shortest path between the source and destination host. If a path exists, it will proceed to properly configure the switches that are part of both the primary and the backup paths, adding, in a hash table, the ID of the path, stored as "src-dst" and the actual path among those two hosts.

Following the Link Up or Link Down API, the controller will check if those requests carry some relevant information for the graph (i.e.: information that it has not received yet), and, if the graph is modified, it will trigger a re-computation of the path on each path saved to the hash table.

The link up API, moreover, can be used to notify a new node joining the network. When this happens, indeed, the switches will inform the controller of the new neighbor, and it will add it to the graph used for path computation.

5.4. Path computation and switches configuration

The SDN controller has to provide a specific configuration, allowing the P4-switches to efficiently manage link failure events even if they are unreachable from the controller itself. Moreover, it is worth considering that the switches are not aware of the overall network, so they need to make decisions according to local knowledge. As an example, let us consider Fig. 6.

If the controller is requested to compute the path between H1 and H2, the result will be [S1 - S2 - S3 - S4]. Then, each switch of the network must also have a backup link. To compute the backup link for each switch, the controller considers a graph where the next hop of the switch is not present and computes a new path from the switch considered to the destination also checking if it might be convenient to send back the packet on the ingress interface. In case this is not possible, the controller tries by deleting only the link between the current switch and the next hop. In this way, the backup path can still involve the next hop but not the next link. This kind of backup works fine if a link fault occurs, but can do nothing in case of node failure. This approach is less robust, and therefore, it is used by the

controller only if the first method does not return any viable options. If none of the described mechanisms find any alternative, a backup simply does not exist. In any case, as stated previously, if one of the switches detects a new link, a notification is sent to the controller, which can recompute new primary and backup paths, ensuring that, if a backup exists, a switch is configured to use it in case of failure. The found backup will be a sub-optimal solution of the shortest path if compared with a path computed after the failure happens. At any rate, after a fault is detected and notified to the controller, it will provide the switches with updated flow rules, ensuring the suboptimal solution will be used for the shortest possible amount of time. After the backup path has been found, it is necessary to configure the switches on this path also. If the only available alternative as backup involves sending the packet on the ingress port (e.g.: The switch S3 in Fig. 6), it is necessary to manage this circumstance efficiently in order to avoid packets that are forwarded on a switch and send back immediately. For this purpose, we deployed on the P4 switches an action called “*set_deprecated_primary*”. Whenever a packet sent on an interface comes back on the same interface, the switch sets the primary port as deprecated and starts using the backup. The node, in this case, assumes that, if a packet comes back on the egress interface, a link on the primary path must be unusable; it thus sends the packets and all the next ones on the backup interface. This action allows the switch to make decisions about the routing even if it cannot detect any link or node failure, reducing the latency and the traffic load.

sudo systemctl disable NetworkManager-wait-online.service, and it is not in turn protected. Therefore, after a failure on the primary path, the communications go through a period with no protection. However, the controller reconfigures the primary and backup paths as soon as possible after a failure, reducing the unprotected window to the bare minimum.

In the following, we describe of some of the methods we implemented to run the SDN Controller. The first method implemented is the **configSwitch**. It is needed to find the shortest path between two endpoints, using the Dijkstra function described in the previous subsection. It then, for each switch on the shortest path, finds the ports among the nodes using the info stored in the graph and a backup port using the function in Algorithm 2. Eventually, the controller will write the rules in the flow table of each involved switch. The function **configSwitch** needs only to know the source, the destination and the destination UDP Port. Once it finds the shortest path, each switch on it will be properly configured.

In the following, we also show the pseudo-code of the function used to find the backup port for each switch on the primary path (Algorithm 2). This function uses a copy of the main graph where it deletes the next node on the primary path and then proceeds to find the shortest path from the given node. The input parameters needed are the following:

- the index of the switch for which we want to find the backup port on the primary path;
- the primary path;
- the destination port;
- the graph.

As shown in the snippet, once the controller finds an alternative path starting from the *i*th switch on the shortest path, it proceeds to set the proper table entries for each involved switch.

Therefore, the configuration algorithm uses Dijkstra not only to find the shortest path between source and destination but also on each switch of the shortest path. The complexity of the configuration algorithm is $|path| \times O((V + E) \times \log_2 V)$, where:

- $|path|$ is the number of the node on the shortest path
- $O((V + E) \times \log_2 V)$ is the time complexity of the Dijkstra algorithm. Particularly, *V* and *E* are respectively the number of nodes and edges of the graph representing the network.

Algorithm 2: configbackup

```

1 Input: i, path, dstPort, graph
2 backupGraph ← copy(graph)
3 curr ← backupGraph.getVertex(path[i])
4 next ← backupGraph.getVertex(path[i+1])
5 dst ← backupGraph.getVertex(path[-1])
6 src ← backupGraph.getVertex(path[0])
7 backupGraph.delVertex(next)
8 backupPath ← shortestPath(curr, dst, backupGraph)
9 if dst.getDistance = ∞ then
10   backupGraph ← copy(graph)
11   backupGraph.delEdge(curr,next) ▷ If no backupPath found,
   retry deleting only the link
12   backupPath ← shortestPath(curr, dst, backupGraph) if
   dst.getDistance = ∞ then
13     return NULL
14   end
15 end
16 backupPort ←
   curr.getPort(backupGraph.getVertex(backupPath[1]))
17   ▷ Set table entries for switch on backup path
18 ConfigPath(backupPath)
19 return backupPort
  
```

In the worst-case scenario, $|path| = V$, so the time complexity of the algorithm is: $O(V \times (V + E) \times \log_2 V)$. From this first arithmetical analysis, it results that the algorithm has a polynomial complexity. In the evaluation section, we will experimentally validate this result by checking the elapsed time of the computation on networks of different sizes. Both the functions described in this section use a third function needed to write the table entries on the P4-switches. This function uses the library *p4runtime.sh*. At the bootstrap, the controller initializes a shell through each switch of the network and then stores the reference to that shell in a Python dictionary. This dictionary, moreover, also contains details about the node of the network, such as the MAC address, IP address, and path of the JSON file with the P4 pipeline.

The same function will take also care of the action “*set_deprecated_primary*”. It indeed will also write an action in case a packet with the same source, destination and destination port as the one forwarded will show up at the primary port set for that packet. In this case, the action will set the new egress port as the destination port.

6. Controller scalability and capacity test

In order to evaluate the size of the network the controller can handle, we have carried out Non-Destructive tests [22] including scalability and capacity tests.

The edge server where the controller has been deployed has the following properties:

- 16 core Inter Xeon Gold 2.20 GHz, with hyper-threading (2 thread per core);
- 77 MiB L3 Cache;
- 8 MiB L2 cache;
- 512 KiB L1 cache;
- 64 GiB RAM (4 × 16 GiB DDR4) with 0.3 ns access delay;
- 2 SDD of 479 GB for storage;
- OS Ubuntu LTS 20.04.06 with kernel Linux 5.15.0-87-generic.

To test both the scalability and the capacity of the deployed architecture we created a synthetic workload using Apache JMeter [23], a traffic generator that allows the users to forge packets in several formats and retrieve target metrics like throughput, latency and number of errors and/or timeout requests. The workload provides a growing

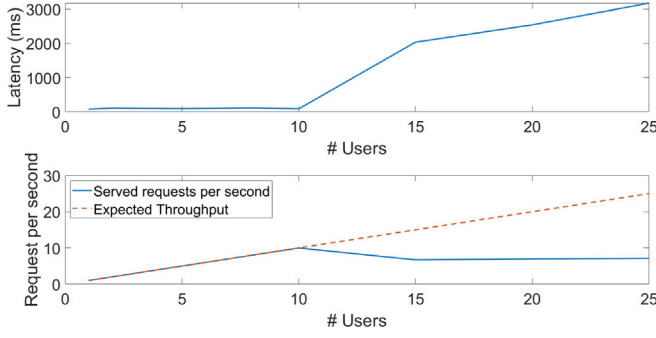


Fig. 7. Capacity test results.

set of threads that send one request per second to the controller. Those requests force the controller to recompute the path between a give source and destination pair of hosts. At the beginning, just one thread is instantiated. Then, as the test suite proceeds, it is re-run with a growing number of threads up to 25, thus introducing into the controller a higher computational load. Each thread sends one request per second for 30 s. The network is simulated and it is composed by 51 switches. Two switches work as AP (e.g., AP1 and AP2) for the two hosts. Then, AP1 and AP2 are interconnected by a squared topology composed by 7×7 switches. The aim of this test is to saturate the computational power of the node and find out the maximum allowed load. Also, this test can be useful to find out how resilient is the controller to DDoS attacks [24].

In Fig. 7 the test results are reported. We observe that the maximum allowed capacity is 8 requests per second. In case of heavier load, the requests are queued, producing an increase of latency without improving the number of requests served per second. In Fig. 8 we show the resource utilization while the number of threads sending requests increases. We can see that, while the number of threads grows, the CPU utilization also increases. After 8 requests per second, the computational capacity is at 6%. This corresponds exactly to one core processor. Since the controller is a sequential process, its execution cannot be split among several cores. Thus, starting from 10 requests per second, the resource utilization reaches its limit. It is also interesting to notice how, at 10 requests per second, the number of interrupts and context switches increases sharply. Indeed, since the capacity of the core is fully taken by the controller process, the operating system needs to continuously perform context switches to run other parallel tasks that require the CPU. Those frequent context switches lead to a degradation of the performance, which is visible in Fig. 8. Thus, we can pinpoint the processor as the bottleneck of the execution.

The controller should not receive more than 6 or 7 path computation requests per second. Even though this value does not seem to be very high, it is doubtful that a network of limited size have so many topology changes per second; we can state this value satisfies our computational needs. It is important to highlight that the number of update requests can largely vary based on the size of the FANET and its scope. Anyway, we must consider that, with this capacity, it is not hard to generate a DDoS attack on the controller. These results imply the need for further countermeasures to prevent DDoS attacks [25]. The values we find, however, are strictly related both to the computational power of the controller node and to the size of the network. In order to see how the size of the network affects the computational time, we also carried out a scalability test, where we modified the number of switches of the network emulated and sampled the response time of the controller to perform a path computation between two hosts. For the scalability test, we considered networks of different sizes and checked the time that the controller needed to compute a path. We considered the network growing in depth and width, where the depth is the number of hops needed to go from host1 to host2. With width

Table 2

Elapsed time for path computation in ms.

		Depth						
		1	2	3	4	5	6	7
Width	1	27.1	38.3	49.2	60.0	71.9	83.2	94.9
	2	21.6	27.6	33.7	40.6	47.2	54.0	61.0
	3	21.8	28.1	35.4	42.6	50.3	58.3	66.7
	4	21.9	29.3	37.1	46.2	54.1	64.1	74.6
	5	22.2	29.8	38.5	48.2	59.5	72.6	83.8
	6	21.9	31.6	41.2	52.7	66.0	80.8	97.2
	7	22.4	33.2	44.0	58.0	74.2	92.2	112.5

we intend the nodal degree of the switch. The topologies used for the scalability test are summarized in Fig. 9

In the first stage, the depth and width grow from 1 node to 7. The smallest test is on a network formed by only one node, and the largest one is on a network of 49 switches. To provide a quantitative analysis of how much those two factors affect the elapsed time, we created a Design of Experiment (DoE) followed by an Analysis of Variance [26] (ANOVA test), particularly, we ran a full-factor Design of Experiment. Moreover, to differentiate the effect of the combination of the factors from the effect of the error, we repeated the test 5 times per combination. This means a total of $7 \times 7 \times 5 = 245$ executions. In Table 2 we show the average of the elapsed time for each combination of width and depth over the five repetitions.

From Table 2 we notice that for this particular implementation, the computational time is very sensitive to the length of the path. Indeed, the algorithm is based on the idea of finding a new path for each node on the primary path. This means that the Dijkstra algorithm must be iterated d times, considering d as the number of nodes along the shortest path. At the same time, no growing pattern is clearly visible in the width of the graph; therefore, we decided to re-run the test with a growing number of nodes. In this case, we tested the network for a width of up to 20 nodes and a depth of up to 25 nodes, for a total of 500 nodes.

On large networks, as shown in Figs. 10(a) and 10(b), the nodal degree also has a relevant impact.

Consequently, we perform an analysis to find out how much variance the different factors explain in the overall model. We define as y_{ijk} the response variable of the experiment (i,j,k) , corresponding to the elapsed time to compute a path in a network with $width = i$, $depth = j$ at the k th repetitions. As a consequence of that, we obtained the equation in (1) where:

- μ_0 is the constant component of the experiment, obtained as the average of all the results (2);
- α_i is the contribution of the factor “width” at the level i , obtained as the average of the response time over j and k minus the constant component (3);
- β_j is the contribution of the factor “depth” at the level j , obtained as the average over i and k minus the constant component (4);
- γ_{ik} is the contribution of the joint effect of the factor width at level i and factor depth at level j , obtained as the average over k of the response time minus the constant component and the contribution of width at level i and depth at level j (5);
- e_{ijk} is the contribution of the error of the experiment (i,j,k) , and is the residual part of the response time once all the previous contributions are eliminated (6).

$$y_{ijk} = \mu_0 + \alpha_i + \beta_j + \gamma_{ij} + e_{ijk} \quad (1)$$

$$\mu_0 = y_{...} \quad (2)$$

$$\alpha_i = y_{i..} - \mu_0 \quad (3)$$

$$\beta_j = y_{.j.} - \mu_0 \quad (4)$$

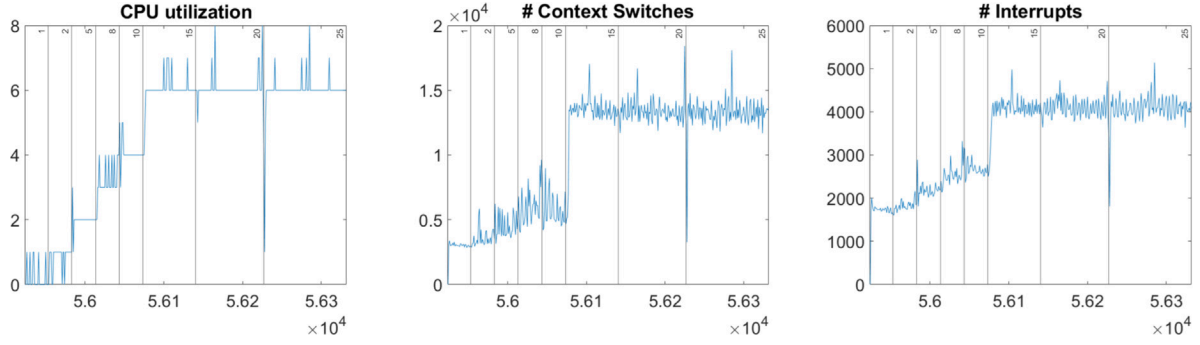


Fig. 8. Resources utilization while the workload increases.

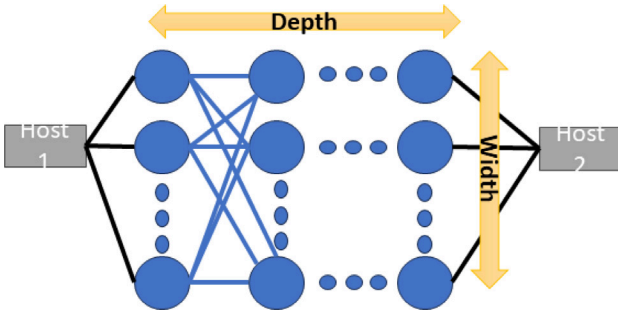
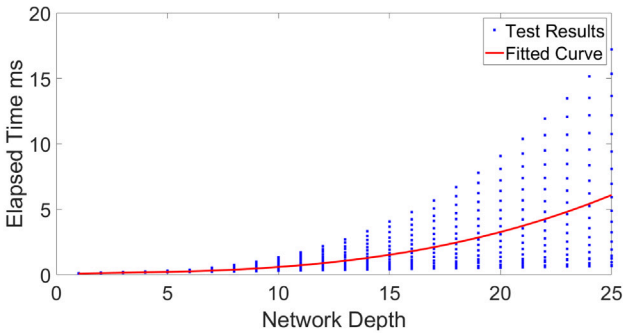
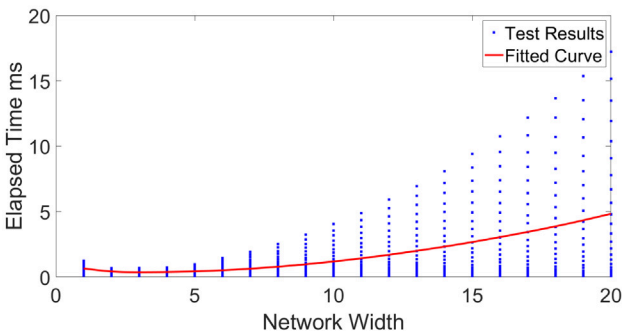


Fig. 9. Topology used for scalability test.



(a) Elapsed time for path computation to the variation of depth



(b) Elapsed time for path computation to the variation of width

Fig. 10. Scalability test results.

$$\gamma_{ij} = y_{i,j} - \mu_0 - \alpha_i - \beta_j$$

(5)

$$e_{ijk} = y_{ijk} - \mu_0 - \alpha_i - \beta_j - \gamma_{ij}$$

(6)

Once we find the impact of each factor at each level, we can calculate the sum of squares as:

$$SS_0 = \sum_{i=1}^{25} \sum_{j=1}^{20} \sum_{k=1}^5 \mu_0 = 25 \times 20 \times 5 \times \mu_0 = 8131 \quad (7)$$

$$SS_A = \sum_{i=1}^{25} \sum_{j=1}^{20} \sum_{k=1}^5 \alpha_i = 20 \times 5 \times \sum_{i=1}^{25} \alpha_i = 8023 \quad (8)$$

$$SS_B = \sum_{i=1}^{25} \sum_{j=1}^{20} \sum_{k=1}^5 \beta_j = 20 \times 5 \times \sum_{j=1}^{20} \beta_j = 4961 \quad (9)$$

$$SS_{AB} = \sum_{i=1}^{25} \sum_{j=1}^{20} \sum_{k=1}^5 \gamma_{ij} = 5 \times \sum_{i=1}^{25} \sum_{j=1}^{20} \gamma_{ij} = 6222 \quad (10)$$

$$SS_E = \sum_{i=1}^{25} \sum_{j=1}^{20} \sum_{k=1}^5 e_{ijk} = 0.732 \quad (11)$$

At this point, we estimated the quality of the model that we built by calculating the R^2 as the ratio between the sum of squares of the regression (SSR) and the sum of squares of the treatment (SST), where $SST = SS_A + SS_B + SS_{AB} + SS_E$ and $SSR = SST - SS_E$. We obtained $R^2 = SSR/SST \approx 1$. This value indicates a model that explains almost all the variance of the experiment, and thus a very robust model.

Since the model approximates the response variable very well, we proceeded to the calculation of the importance of the factors as:

$$depthImp = SS_B/SST = 0.4177 \quad (12)$$

$$widthImp = SS_A/SST = 0.2583 \quad (13)$$

$$jointImp = SS_{AB}/SST = 0.3239 \quad (14)$$

$$ErrImp = SS_E/SST = 3.8066 \times 10^{-5} \quad (15)$$

An F-test performed on the Mean sum of squares also shows that the factors are statistically significant. The results show that the depth explains 42% of the variance of the response time, while the width explains only 26%. This means that the controller can handle topology with a higher nodal degree (i.e.: more link connections) without a strong impact on the response time. Vice versa, the response time sharply increases if the path between two hosts involves several nodes. In the second set of tests, with a bigger network, the impact of the nodal degree, despite being significantly smaller than the impact of the path length, is not negligible. This result is due to the definition of the Dijkstra algorithm itself, shown in Algorithm 3, which involves exploring every node that is reachable from the source node.

The computational time of the shortest path and backup path while varying the number of nodes is shown in Fig. 11.

In Fig. 11, it is shown that the output of the Matlab tools is fitting the experimental results, using a polynomial curve of degree two. This means that the computation time of the controller scales as V^2 , where V is the number of nodes in the network. This is not in contrast with the value calculated in the section *SDN Controller* Section 5.4. The result obtained before indeed is a worst-case scenario and it represents thus

Algorithm 3: Shortest Path Algorithm

```

1 Input: src, dst, graph
2 Output: Shortest Path  $src.distance \leftarrow 0$ 
3  $unvisitedqueue \leftarrow eachinGraph$ 
4  $sort(unvisitedqueue)$   $\triangleright$  sort the queue by distance
5 while  $len(unvisitedqueue) > 0$  do
6    $current \leftarrow unvisitedqueue.pop()$ 
7   for  $n$  in  $current.neighbours$  do
8      $newDistance \leftarrow current.distance + 1$ 
9     if  $newDistance < n.distance$  then
10       $n.distance \leftarrow newDistance$ 
11       $n.previous \leftarrow current$ 
12   end
13 end
14 end
15  $\triangleright$  Build the path using the previous of each vertex  $path \leftarrow [dst]$ 
16  $p \leftarrow dst.prev$ 
17 while  $p \neq Null$  do
18    $path.append(p)$ 
19    $p \leftarrow p.prev$ 
20 end
21 return path

```

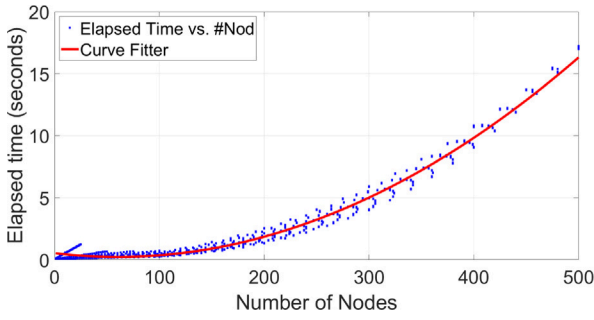


Fig. 11. Fitting Curve of the average computation time.

an upper bound of the computational time. In our test, the path length $|path|$ is much smaller than the number of nodes, indeed we have that $V^2 < (V^2 + E) \times \log_2(V)$

To compare these results with other implementations in the literature, we can focus on the work in [27]. The authors compared the performance of several SDN controllers, like Floodlight, ONOS, Ryu, and many others, on three benchmarks. The results show that the number of flow entries installed per second grows more slowly than linearly. While considering those results, we must also take into account that the topology tested is a fat-tree, which is particularly efficient since it can connect the root node to 128 leaves in just 6 hops. In contrast, our topology is not designed to be that efficient, but on the contrary, it stresses the controller with many switches and many edges among the two endpoints. We can thus assume similar performance for our controller, even though it is able to provide the network with resilience mechanisms.

Looking at Table 2, it is also interesting to notice that, when the width is 1, the computation time is higher. In this case, indeed, the controller, for each node in the path, will try to compute a backup path by deleting the next node in the primary path. Since it does not succeed, it will retry by keeping the node and deleting the edge from the graph. Thus, for each node, it will recompute twice the backup path.

7. Experimental results

The testing of the architecture is organized as follows:

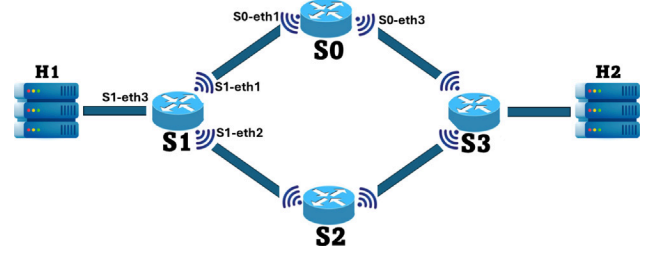


Fig. 12. P4-UAV topology in Mininet for steering validation.

- In section A, we validate the steering mechanism in a virtual environment (Mininet). This step is necessary in order to check if the P4 code works as expected.
- Afterwards, in section B, we describe the setup for the test on a real testbed;
- In section C, we present the functional validation of the controller. In this phase, we test that the controller is able to properly compute the paths and configure the P4-Switches.
- In section D, finally, we present the test results. Here we show that, after the configuration, the network is able to work also in conditions of controller fault, with a lower unavailability time compared to a classical SDN implementation.

7.1. Steering functional validation

The validation was conducted on both the actual topology of the P4-UAV nodes and within the Mininet emulation environment. This emulator allows for the customization of topologies and the replication of P4-UAV nodes using the same compiled P4 code and the same set of flow rules applicable in the real topology.

The functional validation of P4 steering was performed using the Mininet topology outlined in Fig. 12, where two hosts, H1 and H2, designated respectively as the transmitter and the receiver, connected through a network composed of four P4 switches denoted as S0, S1, S2 and S3. H1 is connected to S1 switch while H2 is connected to S3 switch.

A One-way UDP traffic (from H1 to H2) is considered as the traffic flow. For node S1, the (S1-S0-S3) path has been set as the primary route, while the (S1-S2-S3) path is set as the backup route. Furthermore, the interface 'S1-eth1' has been set as a deprecated port in case of a bounce-back event. Regarding node S0, the interface 'S0-eth3' has been configured as the primary output port and interface 'S0-eth1' as the backup port (i.e., routing the outgoing packet on the receiving interface).

In the first experiment, all the interfaces are up (e.g., all the fields in the interface_status register is set to 'UP'). Wireshark captures, depicted in Fig. 13, conducted on switches S0 and S1, illustrate that the packets received on S1-eth3 are forwarded along the primary path (S1-eth1 and S0-eth3).

In the second experiment, the S1-eth1 interface is disabled by setting the status to 'DOWN' in the corresponding field of the interface_status register. Wireshark captures, demonstrated in Fig. 14, carried out on the switches S1 and S0, show that the packets received on S1-eth3 are routed towards the backup path (S1-eth2) and no packets are detected on switch S0, confirming the successful execution of the steering process.

In the third experiment, the S0-eth3 interface is disabled by setting the status to 'DOWN' in the relevant field of the interface_status register. Wireshark captures, depicted in Fig. 15, performed on the switches S0 and S1, illustrate three traffic packets relating to three different flows received on S1-eth3. In particular, the first packet with destination port 12345 is sent to switch S0 through interface S1-eth1 (the primary port). Once it reaches S0, where the primary interface S0-eth3 is down,

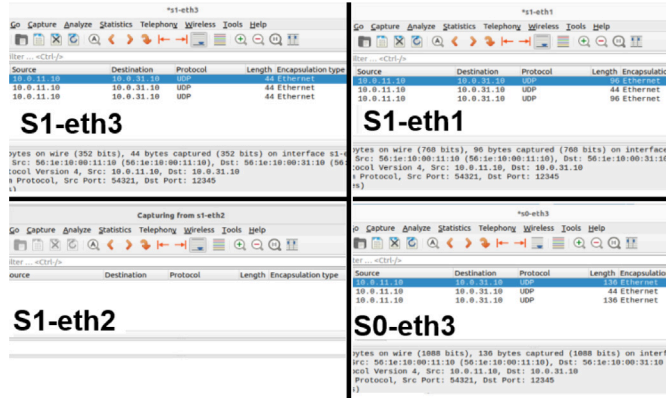


Fig. 13. Steering on the primary path with up interfaces.

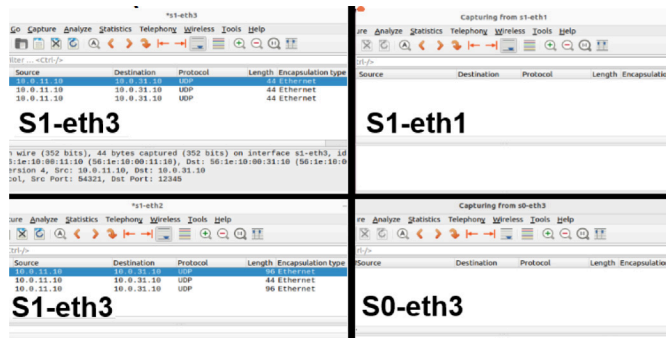


Fig. 14. Steering on the backup path with primary interface down.

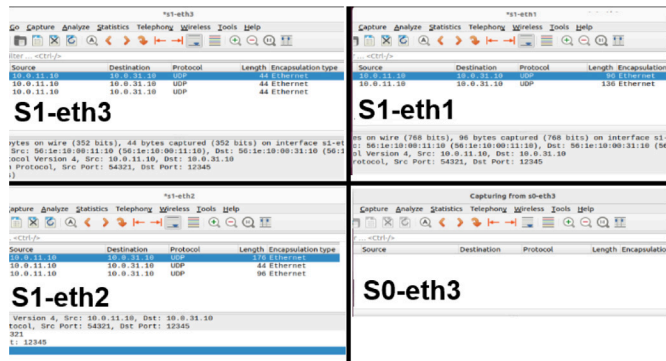


Fig. 15. Steering with bounce back and stateful updating of the primary interface deprecated.

the packet is rerouted back towards S1 using the receiving interface (S0-eth1). At this point, the status of interface S1-eth1 is updated to 'UP-DEPRECATED' and this packet (with destination port 12345) is routed via the backup route (S1-eth2). All the subsequent packets arriving from H1 on S1-eth3, are automatically redirected to the backup path. This behavior confirms the successful implementation of this steering policy. In this example, since the packet generation rate is very low (1 packet per second), the bounce-back effect is applied to only one packet, and the mechanism guarantees that the bounce-back time interval is minimum, equivalent to the sum of twice the transmission time, added to twice the propagation time, alongside the processing duration at the P4 switch where the failure takes place.

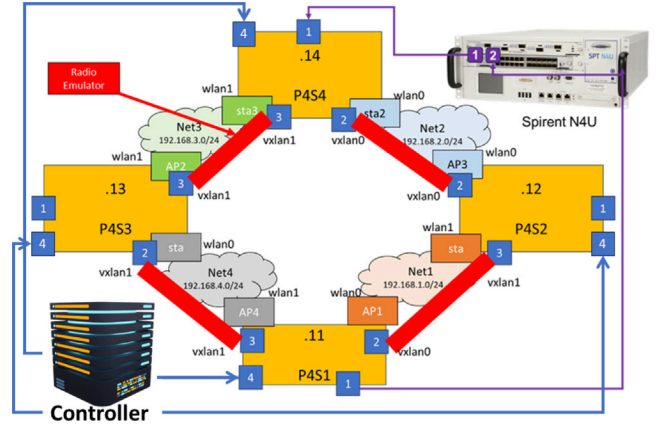


Fig. 16. Schema of Testbed.

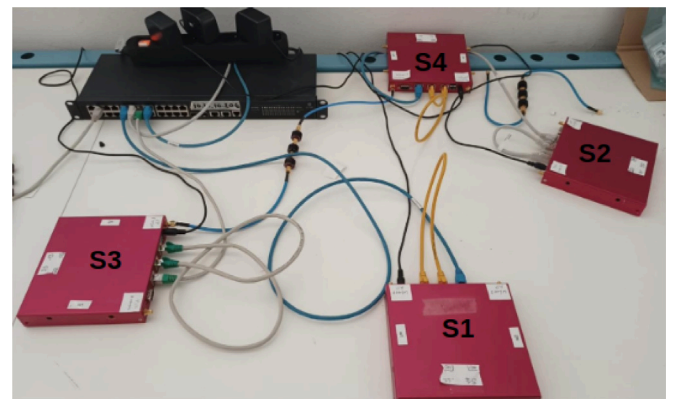


Fig. 17. Laboratory Testbed.

7.2. Testbed description

The proposed solution has been deployed and evaluated in a real testbed utilizing the topology depicted in Fig. 16.

The topology is composed of 4 switches, where .14 is the source and .11 is the destination. With this topology, we can test the ability of the controller to find a primary and a backup path. The controller is deployed on a dedicated node. A Spirent N4U Traffic generator [28] is used to generate traffic and analyze the packet loss. In our scenario, the Spirent sent 1000 packets per second. With this choice, one lost packet corresponds to one millisecond of unavailability of the network.

The elements of the network are P4-enabled devices with an architecture that is similar to the one described in [29]. The implementation of the schema in Fig. 16 is shown in Fig. 17.

The devices used for the test topology are built on an x86 board COTS called APU1 which guarantees compatibility with a wide range of software applications. These devices are equipped with the following components:

- Bord X86 embedded;
- CPU AMD G T40 series, dual-core 1 GHz;
- 2 GB DRAM DDR
- 3 interfaces gigabit ethernet (Realtek TRL8111E)
- 1 Serial port DB9;
- 3 mCPI slots;
- SSD SATA 64 GB

Particularly, the CPU supports both 64 and 32-bit applications, and it is equipped with an L1 64 Kb cache (of which 32 Kb for instruction

```

Dumping entry 0x0
Match key:
* standard_metadata.ingress_port: EXACT      0002
* ethernet.srcAddr      : EXACT      000db940dc56
* ethernet.dstAddr      : EXACT      000db940db86
* udp.dst_port          : EXACT      3039
Action entry: MyIngress.set_steering_state - 01, 02, 01, 01
*****

```

Fig. 18. Table entries of the P4 switch .12.

and 32 Kb for data) and 512 Kb of L2 cache per core. Given the absence of a video board, the standard output has been redirected to the serial port for management, even in the absence of network connectivity. The SSD has been mounted on one of the three mPCI. This storage is used to install an Ubuntu OS and to store the scripts we deployed for the testbed. The boards have been enhanced with two radio modules Qualcomm DAXA-F1, connected to two mPCI slots. Each module supports the 802.11 a/b/g/n/ac standards.

The testbed uses the radio modules to establish connections with the standard 802.11a at 5 GHz. Moreover, the architecture uses the dockerized BMv2 instance [30] for the P4S instances. This approach brings several advantages, like the isolation between the software environment and the underlying OS, the software stability, the scalability, and an optimized deployment process.

7.3. Functional validation of the testbed

Before proceeding with the test on the robustness and recovery capability of the architecture, we needed to validate the testbed in order to avoid unexpected behaviors. Thus, after the connection of the controller to the switches, we checked that the topology handler module, which is responsible for the link discovery and the API call to report the presence or the disruptions of a link, correctly reports this kind of event.

In the next step, we verify the capability of the controller to push flow entries in the P4-switches after a path computation. In Fig. 18 we show an example of table entries for the switch .12.

As we can see, the table entry uses as match parameters (i) the ingress port (value of 0002 in the figure), (ii) the source and (iii) the destination MAC addresses and (iv) the UDP destination port (value 3039 in the figure). The action of the entry includes: (i) the primary port where the traffic is forwarded (value 01 in the figure) and (ii) the backup port (value 02 in the figure). In this case, the backup port is the same as the ingress port. In fact, if its adjacency with .11 goes down, switch .12 sends back the packets to the ingress port, where switch .14 will forward the traffic to switch .13. This is possible because switch .14 has is configured with a table entry with the values of Fig. 18 with ingress port 02, and output port 3. This last table entry comes as a result of the configuration performed by the controller in the Algorithm 2

7.4. Test results

In this section, we will show the experimental results of the tests performed on the P4 SDN FANET. The first step of the controller is to configure the P4-enabled switches with the correct pipelines containing the action programmed through the P4 language. Moreover, once a path computation is performed, it must write the proper flow entries in the table of each switch.

Therefore, the firsts tests results focus on the bootstrap operation of the controller. In Table 3 we show the time in nanoseconds that the controller needs to run the aforementioned operations.

In the table, it emerges that some switches are configured more than once. Indeed, if we consider the configuration of the primary path (.14 - .12 - .11), the controller will also configure the backup for both cases the switch 14 loses connectivity with the switch 12 (.14 - .13 - .11), and the switch 12 loses connectivity with the switch 11 (.12 - .14 - .13 - .11). Therefore, the switches 14 and 11 are configured three times, while the

Table 3

Configuration time of the P4-switches in nanoseconds.

Operation	Switch ID	Writing time
Pushing P4 pipeline	.11	53 366 359
	.12	60 108 452
	.13	65 691 863
	.14	57 417 077
Config primary path	.14	2 420 268
	.12	2 158 798
	.11	2 003 760
Config backup from .12	.14	4 481 632
	.13	2 880 731
	.11	4 393 170
Config backup from .14	.13	2 443 661
	.11	2 496 740

Table 4

Dropped frames for path sw4 to sw1.

Backup	Tx frames	Rx frames	Dropped frames
No	32 505	31 312	1193
Yes	42 903	42 030	873
Yes (no det)	30 249	30 249	0

switches 12 and 11 are configured twice. The configuration time is one order of magnitude lower than the time for pushing the P4 pipeline and is thus negligible if compared with the latter. That said, in our solution, the P4 pipelines are written only once at the bootstrap, and thus, they have no impact on the overall performance. The path configurations, which must be done after every fault, are much faster. Nevertheless, we have to consider that the number of path configurations grows polynomially with the number of switches. As the network grows, this contribution is no longer negligible. Indeed, in the first switch of a path of length N switches, the backup will involve at least N switches (if fewer, this path would be the primary), in the second switch, the backup path will involve at least $N-1$, and so on. So the minimum number of switches to configure is $(N + N - 1 + N - 2 + \dots + 2 + 1) = N(N + 1)/2$. In a more unfortunate scenario, we might configure N^2 switches. From the test, we can consider a median value (more robust to the outliers) for the configuration time of 2.4 ms. With this value, we obtain that in a path with a length of 5 switches, the configuration time is between 36 ms and 60 ms. If we consider a path of length 10, the configuration time is between 132 and 240 ms. Finally, in a path of length 15 switches, the configuration time is between 288 and 540 ms. Overall, we consider those values being satisfying for a FANET of average size.

After the test of the performances for the configurations, we carried on the test to check the recovery performances. In Table 4 are shown the metrics retrieved by the Spirent test generator. The *dropped frames* value is the packet loss experienced when the link between .14 and .12 (from Fig. 16) is unavailable.

The first entry of Table 4 shows the packet loss when no backup port is set. In this case, the overall packet loss (value of 1193 in the table) includes different contributions: the failure detection time, the communication with the controller, the time for the path recomputation and reconfiguration. When the backup port is set (second entry in Table 4), the recovery time is reduced, observing the loss of 873 packets. In fact, in this second case, when a failure is detected, the P4-Switch 14 writes in the *interfaces_status_register* the value "0" - unavailable. When the status in the register is 0 for a given interface, the traffic that is routed on it, is switched to the backup port. In this case, then, the main contributions to the loss of the packets are the detection time and the time needed for the update of the *interfaces_status_register*. To prove further that the controller computation is not involved in the rerouting when the backup port is set, we run a third test. In this scenario, we directly set the *interfaces_status_register* to 0 (value representing a state down of the port). In the last entry of Table 4, we can see that

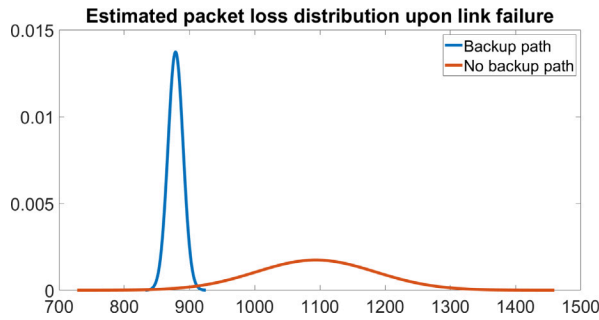


Fig. 19. Packet loss in case of link failure.

the packet loss is 0. Indeed, through this method, we directly set the unavailable state of the interface, without waiting for the detection and eliminating the time contribution of failure detection. The P4-switches thus immediately take action and change the output interface, without losing any packets. This is a significant experimental result that confirms that our steering mechanism is able to minimize the local recovery procedure by eliminating failure notification time and failure recovery time.

We repeated the tests 10 times and obtained the following results:

- A t-test rejects the null hypothesis with a p -value of 0.0156, confirming that the distributions are different;
- The confidence interval of the time distribution of the recovery time with backup path at 90% of confidence is [859.12–898.20], while the same confidence interval for recovery of unprotected path is [939.5–1247.8], showing no overlapping with this given confidence.

Finally, we show in the Fig. 19 a plot of the estimated packet loss distribution for a traffic flow of 1000 packets per second, in case of link failure both in the case of presence of backup path, and in its absence.

8. Conclusion

In this paper, we presented a solution that clearly improves the availability of an SD-Fanet. Indeed, not only have we managed to reduce packet loss whenever a link failure happens, but we must also consider that the network is able to mitigate this kind of event when the controller cannot reach the devices involved.

Furthermore, we also demonstrated that the packet loss that we experienced is entirely due to the detection time that is managed at the kernel level while the handling of the failure is then immediate. Failure detection time cannot be totally eliminated since it depends strictly on the interface technologies and the node operating system kernel, but it can be optimized by resorting to specific hardware-based polling or asynchronous signaling mechanisms.

Our solution aims to apply SDN to FANET, providing, at the same time, a level of dependability whenever a node is no longer reachable by the controller. The control plane is still managed by the controller, but at the same time, it provides the network devices with backup actions that can be taken when a reconfiguration is needed. These actions are taken locally, leveraging the flexibility of the P4 language. The result is a mitigation of the time needed to handle failure events.

This improvement can potentially allow an SDN to become suitable also for applications that have strict safety requirements and thus cannot rely on a centralized controller.

8.1. Limitations and future deployment

In the current implementation of the P4-switches, when a port is declared “UP_DEPRECATED”, it will remain in this state until the controller reconfigures the switch. In order to make the network more efficient, we plan to address this issue in our next work. Another possible limitation of the proposed architecture is the number of requests per second that the controller can handle. Even if in a small network we do not expect so many changes in the topology, we must take into account that, in wider networks, the scalability of the controller becomes critical. A simple solution would involve a refactoring of the code using compiled programming languages like C/C++, to avoid the overhead introduced by Python. Although the Dijkstra algorithm is hardly parallelizable, we could use different instances of the process to reach a higher throughput of served requests. Also, it might be possible to deploy parallel controllers with a load-balancer acting as an interface. In this case, the redundancy of the controller may address the complexity of the geographical positioning of the servers, since all the nodes interact with the load-balancer, splitting requests between the two implementations. Furthermore, with this approach, mechanisms like heartbeat may be deployed, making the overall architecture more robust.

CRedit authorship contribution statement

Domenico Uomo: Writing – review & editing, Writing – original draft, Software, Formal analysis. **Andrea Sgambelluri:** Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation. **Layal Ismail:** Writing – original draft, Software. **Francesco Paolucci:** Visualization, Validation, Supervision, Methodology.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Domenico Uomo reports financial support, article publishing charges, equipment, drugs, or supplies, and travel were provided by Sant’Anna School of Advanced Studies. Andrea Sgambelluri reports financial support, article publishing charges, equipment, drugs, or supplies, and travel were provided by Sant’Anna School of Advanced Studies. Layal Ismail reports financial support, article publishing charges, equipment, drugs, or supplies, and travel were provided by Sant’Anna School of Advanced Studies. Francesco Paolucci reports financial support, article publishing charges, equipment, drugs, or supplies, and travel were provided by CNIT. Domenico Uomo reports article publishing charges, equipment, drugs, or supplies, and travel were provided by CNIT. Andrea Sgambelluri reports article publishing charges, equipment, drugs, or supplies, and travel were provided by CNIT. Layal Ismail reports article publishing charges, equipment, drugs, or supplies, and travel were provided by CNIT. Francesco Paolucci reports article publishing charges, equipment, drugs, or supplies, and travel were provided by Sant’Anna School of Advanced Studies. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partially funded by the European Commission Horizon Europe SNS JU DESIRE6G project, under grant agreement No. 101139285, and by the European Union’s Horizon RIA research and innovation programme under grant agreement No. 101092908 (SmartEdge).

Moreover, the work was partially supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of the NextGenerationEU partnership on “Telecommunications of the Future” (PE00000001, program “RESTART”).

Data availability

No data was used for the research described in the article.

References

- [1] S.K. Singh, et al., A comprehensive survey on fanet: challenges and advancements, *Int. J. Comput. Sci. Inf. Technol.* 6 (3) (2015) 2010–2013.
- [2] P. Massam, E. Babulak, A decent service level, *Commun. Eng.* 1 (2003) 34–37, <http://dx.doi.org/10.1049/ce:20030313>.
- [3] P.E. Heegaard, B.E. Helvik, V.B. Mendiratta, Achieving dependability in software-defined networking—a perspective, in: 2015 7th International Workshop on Reliable Networks Design and Modeling, RNDM, IEEE, 2015, pp. 63–70.
- [4] D. Kreutz, F.M. Ramos, P. Verissimo, Towards secure and dependable software-defined networks, in: *Proceedings of the Second ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking*, 2013, pp. 55–60.
- [5] S. Gilbert, N. Lynch, Perspectives on the CAP theorem, *Computer* 45 (2) (2012) 30–36.
- [6] N. Hu, Z. Tian, Y. Sun, L. Yin, B. Zhao, X. Du, N. Guizani, Building agile and resilient UAV networks based on SDN and blockchain, *IEEE Netw.* 35 (1) (2021) 57–63.
- [7] D. Khan, L.T. Jung, M.A. Hashmani, Systematic literature review of challenges in blockchain scalability, *Appl. Sci.* 11 (20) (2021) 9372.
- [8] A. Nadeem, T. Alghamdi, A. Yawar, A. Mehmood, M.S. Siddiqui, A review and classification of flying ad-hoc network (FANET) routing strategies, *J. Basic Appl. Sci. Res.* 8 (3) (2018) 1–8.
- [9] O.K. Sahingoz, Networking models in flying ad-hoc networks (FANETs): Concepts and challenges, *J. Intell. Robot. Syst.* 74 (2014) 513–527.
- [10] M. Khan, I. Qureshi, E. Safi, I. Khan, Flying ad-hoc networks (FANETs): A review of communication architectures, and routing protocols, 2017, <http://dx.doi.org/10.1109/INTELLECT.2017.8277614>.
- [11] A.H. Wheeb, N.A. Al-jamali, Performance analysis of OLSR protocol in mobile ad hoc networks, *Int. J. Interact. Mob. Technol. (IJIM)* 16 (01) (2022) pp. 106–119, <http://dx.doi.org/10.3991/ijim.v16i01.26663>, URL: <https://online-journals.org/index.php/ijim/article/view/26663>.
- [12] K. Singh, A.K. Verma, Applying OLSR routing in FANETs, in: 2014 IEEE International Conference on Advanced Communications, Control and Computing Technologies, 2014, pp. 1212–1215, <http://dx.doi.org/10.1109/ICACCCT.2014.7019290>.
- [13] M.I.B. Azevedo, C. Coutinho, E.M. Toda, T.C. Carvalho, J. Jailton, Wireless communications challenges to flying ad hoc networks (FANET), in: J.H. Ortiz (Ed.), *Mobile Computing*, IntechOpen, Rijeka, 2019, <http://dx.doi.org/10.5772/intechopen.86544>.
- [14] N.L. van Adrichem, F. Iqbal, F.A. Kuipers, Backup rules in software-defined networks, in: 2016 IEEE Conference on Network Function Virtualization and Software Defined Networks, NFV-SDN, IEEE, 2016, pp. 179–185.
- [15] A. Capone, C. Cascone, A.Q.T. Nguyen, B. Sansò, Detour planning for fast and reliable failure recovery in SDN with OpenState, in: 2015 11th International Conference on the Design of Reliable Communication Networks, DRCN, 2015, pp. 25–32, <http://dx.doi.org/10.1109/DRCN.2015.7148981>.
- [16] Y.-Z. Liao, S.-C. Tsai, Fast failover with hierarchical disjoint paths in SDN, in: 2018 IEEE Global Communications Conference, GLOBECOM, 2018, pp. 1–7, <http://dx.doi.org/10.1109/GLOCOM.2018.8647950>.
- [17] O. Tilmans, S. Vissicchio, IGP-as-a-backup for robust SDN networks, in: 10th International Conference on Network and Service Management (CNSM) and Workshop, 2014, pp. 127–135, <http://dx.doi.org/10.1109/CNSM.2014.7014149>.
- [18] L. Ismail, D. Uomo, A. Sgambelluri, F. Alhamed, F. Paolucci, P4 FANET In-band Telemetry (FINT) for AI-assisted wireless link failure forecasting and recovery, *Comput. Netw.* (2024) 110599.
- [19] P. Congdon, Link layer discovery protocol and MIB, 2002, URL: <https://www.ietf.org/rfc/rfc2865.txt>.
- [20] P4runtime-shell, 2016, URL: <https://github.com/p4lang/p4runtime-shell>.
- [21] Flask, 2024, Accessed: January, 2024. URL: <https://flask.palletsprojects.com/en/3.0.x/>.
- [22] Wikipedia, Nondestructive testing — Wikipedia, the free encyclopedia, 2024, <http://en.wikipedia.org/w/index.php?title=Nondestructive%20testing&oldid=1225060309>. [Online; accessed 25 June 2024].
- [23] Apache JMeter, 2024, Accessed: January, 2024. URL: <https://jmeter.apache.org/usermanual/index.html>.
- [24] A. Shueb, T. Chithralekha, Resource management of switches and Controller during saturation time to avoid DDoS in SDN, in: 2016 IEEE International Conference on Engineering and Technology, ICETECH, 2016, pp. 152–157, <http://dx.doi.org/10.1109/ICETECH.2016.7569231>.
- [25] H.D. Zubaydi, M. Anbar, C.Y. Wey, Review on detection techniques against DDoS attacks on a software-defined networking controller, in: 2017 Palestinian International Conference on Information and Communication Technology, PICICT, 2017, pp. 10–16, <http://dx.doi.org/10.1109/PICICT.2017.26>.
- [26] L. Sthle, S. Wold, Analysis of variance (ANOVA), *Chemometr. Intell. Lab. Syst.* 6 (4) (1989) 259–272, [http://dx.doi.org/10.1016/0169-7439\(89\)80095-4](http://dx.doi.org/10.1016/0169-7439(89)80095-4), URL: <https://www.sciencedirect.com/science/article/pii/0169743989800954>.
- [27] L. Zhu, M.M. Karim, K. Sharif, C. Xu, F. Li, X. Du, M. Guizani, SDN controllers: A comprehensive analysis and performance evaluation study, *ACM Comput. Surv.* 53 (6) (2020) 1–40.
- [28] Spirent, 2024, Accessed: January, 2024. URL: https://www.spirent.com/assets/u/spirent_n4u_chassis_datasheet.
- [29] P. Castoldi, A. Sgambelluri, L. Ismail, F. Paolucci, F. Cugini, D. Bowden, Network programmability for smart factory mobile robotics: the SmartEdge project approach, in: 2023 23rd International Conference on Transparent Optical Networks, ICTON, 2023, pp. 1–5, <http://dx.doi.org/10.1109/ICTON59386.2023.10207553>.
- [30] Behavioral model version 2 (BMV2), 2016, URL: <https://github.com/p4lang/behavioral-model>.



Domenico Uomo received his MSc in computer science and engineering from the University of the Studies of Napoli Federico II (2020) with full marks, discussing a research thesis on Real-Time virtualization for railways signaling as a service where he has deepened the studies of safety-critical and real-time architectures. The same year he started working as a data engineer on highly sensitive data. In 2022 he got a scholarship with Scuola Superiore Sant'Anna di Pisa. His research topics involved safety assessment of critical networks and Artificial Intelligence.



Andrea Sgambelluri is an Assistant Professor at the Scuola Superiore Sant'Anna of Pisa, Italy, since 2019. He published around one hundred papers (source Google Scholar, May 2021) in International Journals and Conference Proceedings. In March 2015 he won the grand prize at 2015 OFC Corning Outstanding Student Paper Competition with the paper "First Demonstration of SDN-based Segment Routing in Multi-layer Networks". His main research interests are in the field of control plane techniques for both packet and optical networks, including Software Defined Networking (SDN) protocol extensions, network reliability, industrial Ethernet, switching, segment routing application, YANG/NETCONF solutions for the dynamic management, telemetry, (re)programming and monitoring of optical devices.



Layal Ismail received her B.S. in information and communication technology engineering from University of Tartous (2013), Tartous, Syria. She received her M.S in industrial automation from University of Tartous (2020), discussing a research thesis on wireless data acquisition system for control purposes. In 2023 she got a Research Training Grant at CNIT, Pisa, Italy. Currently, she is PhD student at Scuola Superiore Sant'Anna. Her research interests are software defined networking and wireless networks.



Francesco Paolucci (M) received the Laurea degree in telecommunications engineering from the University of Pisa in 2002, and the Ph.D. degree from Scuola Superiore Sant'Anna, Pisa, in 2009. In 2008 he was granted a research Merit Scholarship at the Institut National de la Recherche Scientifique (INRS), Montreal, Quebec, Canada. Currently, he is Head of Research at CNIT, Pisa Italy. His main research interests are in the field of network control plane, orchestration for edge/cloud platforms, traffic engineering, network disaggregation, advanced network telemetry, SDN/P4 data plane programmability. He has been involved in many European research projects on next generation control networking (BRAINE, DESIRE6G, SMARTEDGE, CLEVER, SEASON). He is co-author of 3 IETF Internet Drafts, more than 200 publications in international journals, conference proceedings and book chapters, and filed 4 international patents. He is Associate Editor of the IEEE/OSA Journal of Optical Communications and Networking (JOCN).